

PhantomRing - Malware Analysis

Your organization's SOC team intercepted a suspicious binary during a routine threat hunting operation on a Linux server. The file was found in /var/tmp with an unusual name and was attempting to establish outbound connections. Initial analysis suggests this could be a post-exploitation agent. Your task is to perform static analysis on the binary to identify its capabilities, extract indicators of compromise, and understand the threat actor's infrastructure.

What is the SHA256 hash of the malicious binary?

Virus total → Details

The image shows the VirusTotal details page for a file. At the top, a circular progress indicator shows a 'Community Score' of 26 out of 64. To the right, a red banner states '26/64 security vendors flagged this file as malicious'. Below this, the file's SHA256 hash is displayed: 2d7b1b2178f76c26893b2a56cbf9b36700235259e76b893d53817d5b66b634a5. The file is identified as 'agent' and has tags for 'elf', 'shared-lib', and '64bits'. A navigation bar includes tabs for DETECTION, DETAILS (selected), RELATIONS, BEHAVIOR, and COMMUNITY (with a count of 2). A banner encourages joining the community. The 'Basic properties' section lists various hashes and file metadata:

Property	Value
MD5	6d9a243b881984aa25fc0e770dc0634c
SHA-1	6580406b3487c7bc32ae8024add3715131f53d24
SHA-256	2d7b1b2178f76c26893b2a56cbf9b36700235259e76b893d53817d5b66b634a5
Vhash	4c1eb79f20c52085430c7634f43e790e
SSDEEP	768:MXVxtldVNF91tldVNF91tldVNF91tldVNoAYwlg4QoAa0dRgmAzw1MI:Mt5Mw1M
TLSH	T1FEE2A31BB291DF38E4D4F2301BDBD6E0A62078F06736315F275546B72AB33984B78A46
File type	ELF executable linux elf
Magic	ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld
Telfback	1122d0e750dd3f028011e12570e72b4f8011486e42727211128520c5d4817421cc085e4e

What is the IP address hardcoded in the binary for C2 communication?

→ 192.168.56.1

What port does the agent connect to on the C2 server?

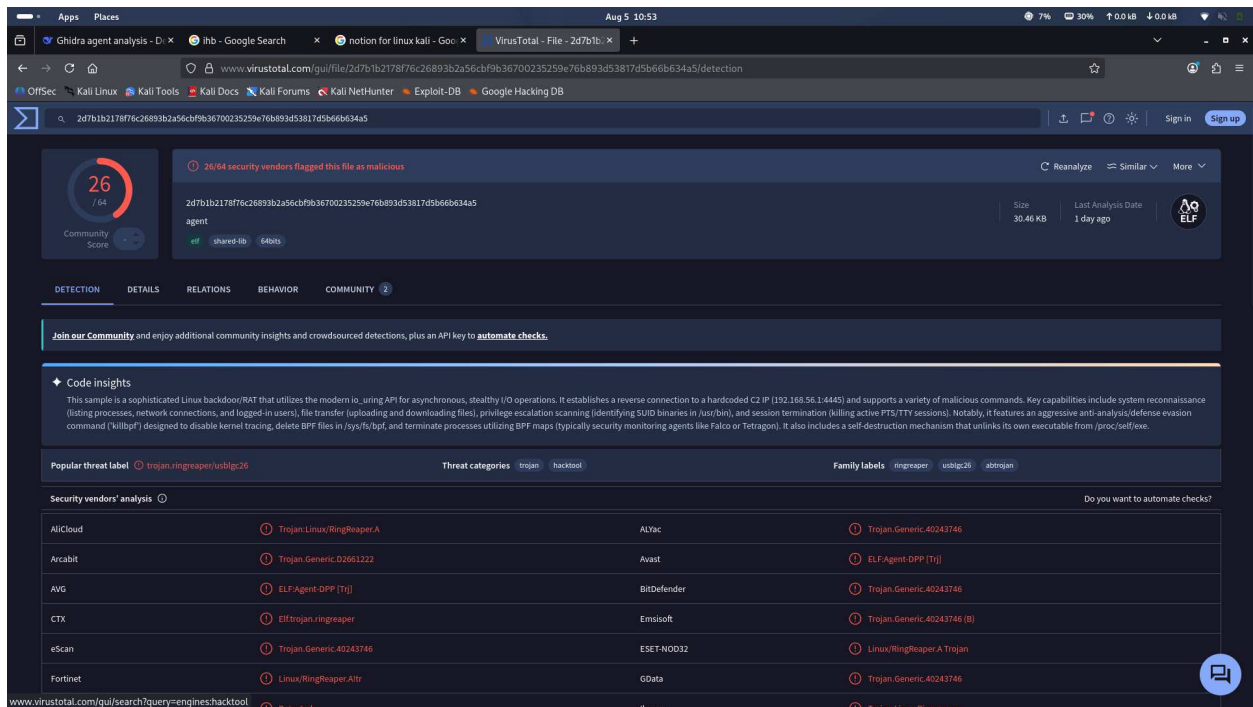
→ 4445

What Linux kernel interface does this malware abuse to evade EDR syscall

monitoring?

→ io_uring

in Detection tab read the code insights - it shows base of the malware - what is actually does



Now enter the Ghidra for other analysis

How many seconds does the agent wait before attempting to reconnect after a failed connection?

→ now we have to find the failed connection to dod that

select - listing tab → ctrl + F → check all fields → search for "connect" → Click search all

Edit Help				
Search Text: "connect" [ProgramDatabase] - (agent) (25 entries)				
Location	Label	Namespace	Preview	
.strtab:000000e6			utf8 u8"io_uring_prep_connect"	
.symtab:00000168			io_uring_prep_connect	
00101758	io_uring_prep_connect	Global	io_uring_prep_connect	
00101758	io_uring_prep_connect	Global	undefined io_uring_prep_connect()	
0010370c	cmd_exit		LEA RAX,[s_Agent_disconnecting_and_exiting_00105308]	
00103713	cmd_exit		MOV qword ptr [RBP + local_20],RAX=>s_Agent_disconnect...	
0010371b	cmd_exit		MOV RDI=>s_Agent_disconnecting_and_exiting_00105308,RAX	
00103726	cmd_exit		MOV RDX=>s_Agent_disconnecting_and_exiting_00105308,q...	
001042b0	main		CALL io_uring_prep_connect	
00104388	main		LEA RAX,[s_+[]_Connected_to_%s:%d_0010556f]	
0010438f	main		MOV RDI=>s_+[]_Connected_to_%s:%d_0010556f,RAX	
001043b2	main		LEA RAX,[s_connect()_failed:_trying_to_reco_00105548]	
001043b9	main		MOV RDI=>s_connect()_failed:_trying_to_reco_00105548,RAX	
00104587	main		LEA RAX,[s_+[]_Connection_closed_00105599]	
0010458e	main		MOV RDI=>s_+[]_Connection_closed_00105599,RAX	
00105308	s_Agent_disconnecting_and_exiting_00105308	Global	s_Agent_disconnecting_and_exiting_00105308	
00105308	s_Agent_disconnecting_and_exiting_00105308		ds "Agent disconnecting and exiting\n"	
00105548	s_connect()_failed:_trying_to_reco_00105548	Global	s_connect()_failed:_trying_to_reco_00105548	
00105548	s_connect()_failed:_trying_to_reco_00105548		ds "connect() failed: trying to reconnect\n"	
00105548	s_connect()_failed:_trying_to_reco_00105548		ds "connect() failed: trying to reconnect\n"	
0010556f	s_+[]_Connected_to_%s:%d_0010556f	Global	s_+[]_Connected_to_%s:%d_0010556f	
0010556f	s_+[]_Connected_to_%s:%d_0010556f		ds "[+] Connected to %s:\n"	
00105599	s_+[]_Connection_closed_00105599	Global	s_+[]_Connection_closed_00105599	
00105599	s_+[]_Connection_closed_00105599		ds "[+] Connection closed"	
001057d0			ddw io_uring_prep_connect	

```

CALL io_uring_prep_connect
LEA RAX,[s_+]_Connected_to_%s:%d_0010556f]
MOV RDI=>s_+]_Connected_to_%s:%d_0010556f,RAX
LEA RAX,[s_connect()_failed:_trying_to_reco_00105548]
MOV RDI=>s_connect()_failed:_trying_to_reco_00105548,RAX
LEA RAX,[s_+]_Connection_closed_00105599]
MOV RDI=>s_+]_Connection_closed_00105599,RAX
s_Agent_disconnecting_and_exiting_00105308
ds "Agent disconnecting and exiting\n"

```

→ select it

```

00104588 08 20 00      MOV     EAX,0x20
001043ad be 01 00      MOV     ESI,0x1
001043b2 48 8d 05      LEA     RAX,[s_connect()_failed:_trying_to_reco_001055... = "connect() failed: trying to r...
001043b9 48 89 c7      MOV     RDI,>s_connect()_failed:_trying_to_reco_001055... = "connect() failed: trying to r...
001043bc e8 ff d0      CALL    <EXTERNAL>:fwrite
001043c1 48 8b 95      MOV     RDX,qword ptr [RBP + -0x10118]
001043c8 48 8d 85      LEA     RAX,[RBP + -0x100f0]
001043cf 48 89 d6      MOV     RSI,RDX
001043d2 48 89 c7      MOV     RDI,RAX
001043d5 e8 9b d2      CALL    io_uring_cqe_seen
001043da 8b 85 e0      MOV     EAX,dword ptr [RBP + -0x10120]
001043e0 89 c7      MOV     EDI,EAX
001043e2 e8 a9 cf      CALL    <EXTERNAL>:close
001043e7 bf 78 00      MOV     EDI,0x78
001043ec e8 ef d0      CALL    <EXTERNAL>:sleep

```

→ here we see the call fuction fwrite that prints the failed

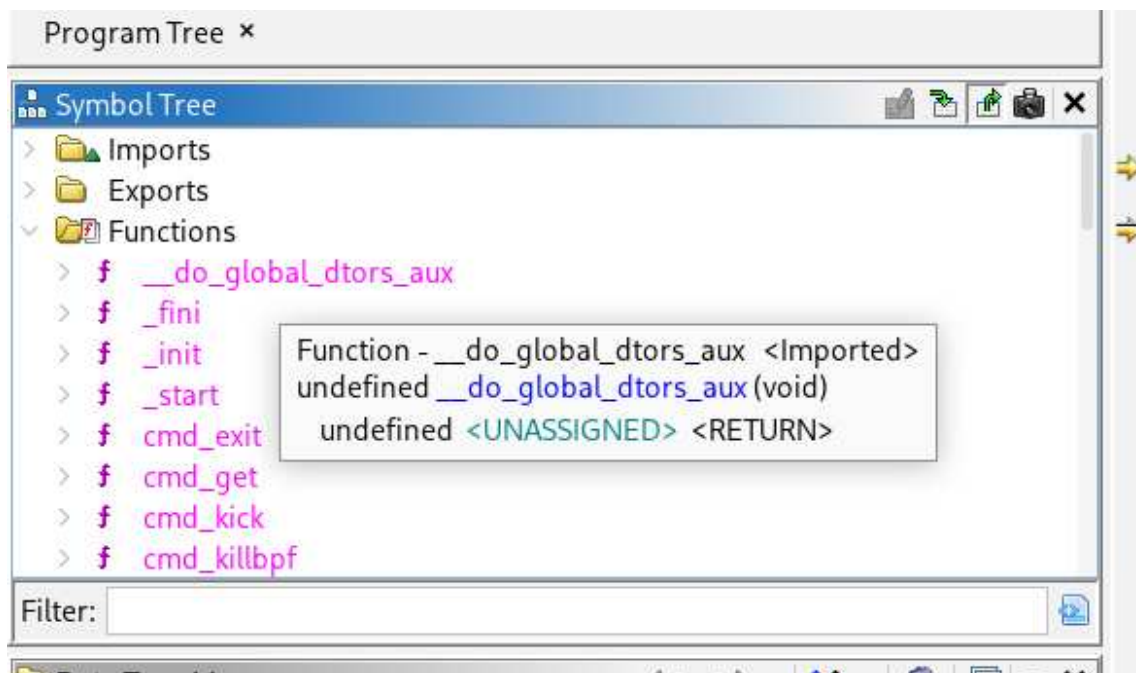
→ Below we see another CALL function close → that clears the memory and closes the failed/ broken socket (network conn)

we see EDI 0x78 in base 16 format - assembly

when we convert it we get 120 sec

Ans : 120

How many different commands does the agent support? (excluding invalid commands)



count how many " cmd_ "

Ans: 9

What file does the agent read to enumerate logged-in users?

```

00105006 00      ??      00h
00105007 00      ??      00h

s_/var/run/utmp_00105008      XREF[1]:      cn
00105008 2f 76 61      ds      "/var/run/utmp"
      72 2f 72
      75 6e 2f ...

s_Error_reading_/var/run/utmp_00105016      XREF[4]:      cn
      cn
      cn
      cn

00105016 45 72 72      ds      "Error reading /var/run/utmp\n"
      6f 72 20
      72 65 61 ...

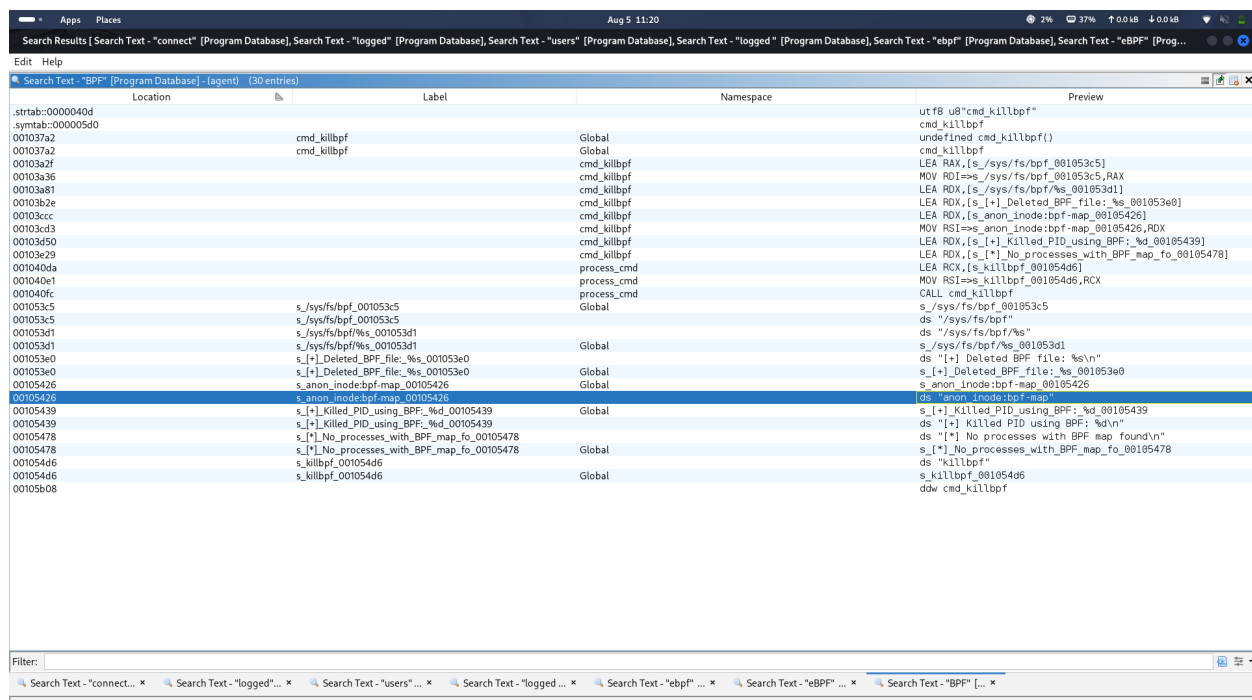
s_Logged_users:_00105033      XREF[1]:      cn
00105033 4c 6f 67      ds      "Logged users:\n"
      67 65 64
      20 75 73 ...

s %-8s %-8s 00105042      XREF[1]:      cn

```

Ans: /var/run/utmp

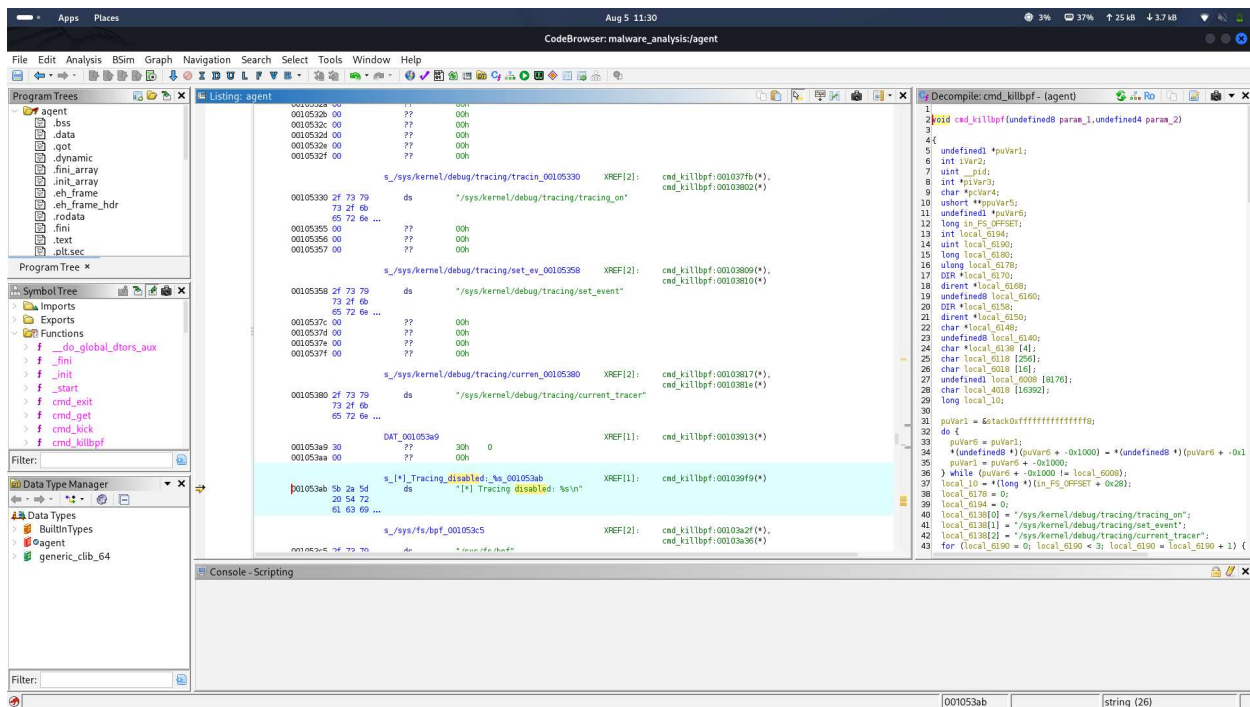
What string does the agent search for in /proc/[pid]/maps to identify security tools using eBPF?



→ Search bpf the hint is strings - look for " "

Ans : anon_inode:bpf-map

What is the full path of the first tracing file the agent attempts to disable?

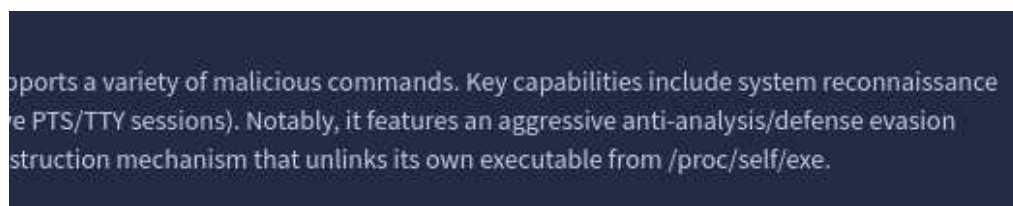


Search → disable static analysis scroll up to the first file path

Ans : /sys/kernel/debug/tracing/tracing_on

What procs path does the agent read to find its own executable location before self-destruction?

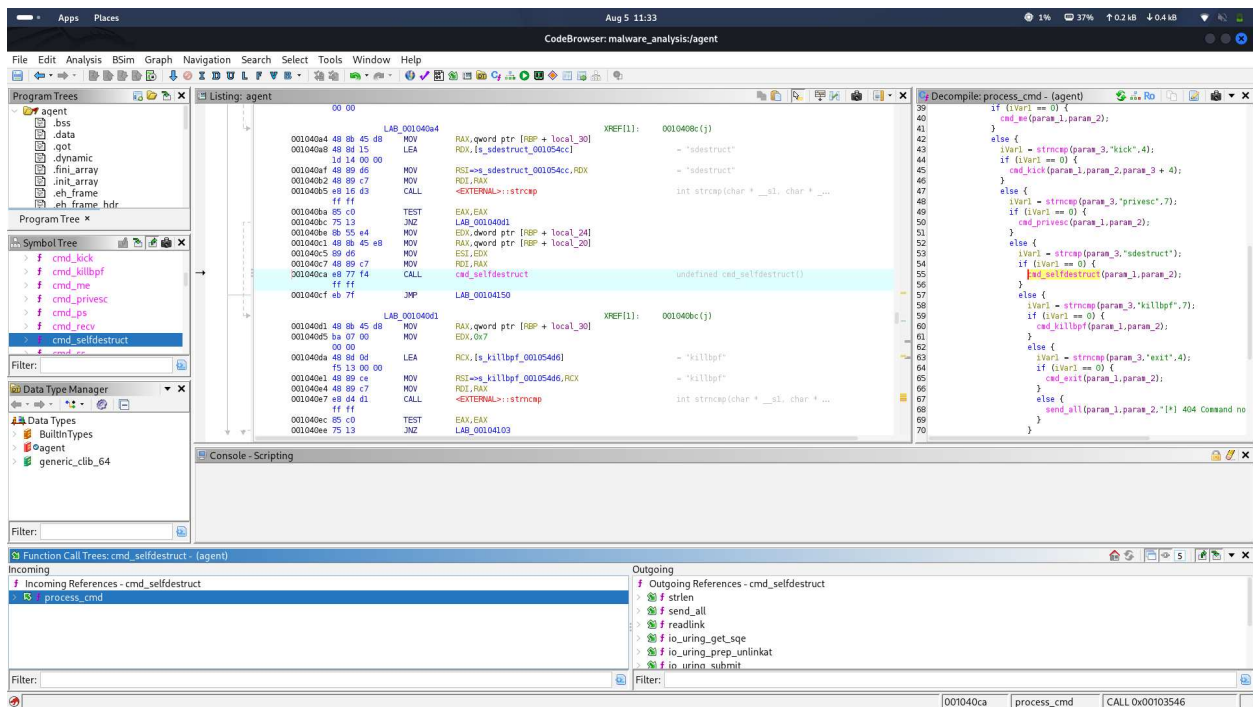
Ans : /proc/self/exe



What command string is compared by the agent to trigger deletion of its own binary?

Ghidra → Symbol tree → cmd_selfdestruct → windows → Click function call tree

we see two tab incoming and outgoing → click process_cmd in incoming see the left decompiler



Ans : sdestruct

This Sherlock lab made me analyse statically, learned few concepts in reverse - ghidra